Detection Engineering Program Proposal

Proposal for Establishing and Operationalizing a Detection Engineering Capability

Prepared for: Cybersecurity Leadership Team

Prepared by: Adam Ring

Version: 3

Date: 2026

Internal Use Only

Table of Contents

Contents

Executive Summary 5

Introduction 6

Why a Detection Engineering Program Is Needed 7

Goals of the Program 8

Detection Engineering Frameworks and Standards 9

Adversary Behavior Frameworks 9

Detection Engineering Methodologies 9

Detection Development Frameworks 10

Security Standards and Guidance 10

Detection Engineering Lifecycle 11

1. Detection Request and Intake 11

2. Research and Analysis 11

3. Detection Development 11

4. Validation and Testing 12

5. Deployment 12

6. Monitoring and Tuning 12

7. Continuous Improvement and Maintenance 12

Detection Engineering Operating Model 13

Security Operations Center (SOC) 13

Threat Intelligence 13

Incident Response 13

Red Team and Purple Team Operations 14

Detection Engineering 14

Detection Engineering Governance (RACI) 15

Responsible 15

Accountable 15

Consulted 15

Informed 15

Detection Engineering Metrics and KPIs 16

Detection Coverage Metrics 16
Detection Quality Metrics 16
Operational Metrics 16
Validation Metrics 17
Program Maturity Metrics 17
Implementation Timeline and Roadmap 18
Phase 1: Program Foundation 18
Phase 2: Detection Development and Coverage Expansion 18
Phase 3: Validation and Adversary Simulation 19
Phase 4: Continuous Improvement and Program Maturity 19
Detection-as-Code Framework Recommendation 20
Detection Rule Standardization 20
Version Control 20
Peer Review and Quality Assurance 20
Detection Rule Lifecycle Management 20
Integration with Security Monitoring Platforms 21
Detection Engineering Maturity Model Assessment 22
Level 1: Reactive Detection 22
Level 2: Structured Detection Development 22
Level 3: Threat-Informed Detection Engineering 22
Level 4: Continuous Detection Validation 22
Level 5: Mature Detection Engineering Program 23
Priority Data Sources for Detection Engineering 24
Endpoint Telemetry 24
Identity and Authentication Logs 24
Network Telemetry 25
Cloud and SaaS Activity 25
Security Platform Alerts 25
Data Onboarding Strategy 26
Identification of New Data Sources 26
Data Source Evaluation 26
Integration into Security Monitoring Platforms 26
Detection Use Case Development 27

Continuous Monitoring and Optimization 27
Purple Team Validation Plan 28
Detection Validation Through Adversary Simulation 28
Collaborative Purple Team Exercises 28
Identification of Detection Gaps 28
Continuous Improvement of Detection Rules 28
Appendix A – Detection Rule Request Template 30
Rule Request Template 30
Appendix B – YAML Detection Rule Template 32
YAML Detection Rule Template 32
Appendix C – Validation Log Template 34
Detection Validation Log Template 34
Example Entry 34
Appendix D – Change Control and Version History 36
Document Version History 36
Change Control Log Template 36
Example Entry 36
Detection Engineering RACI Matrix 37
RACI Definitions 37

Executive Summary

Modern cyber threats continue to evolve in sophistication, persistence, and scale. Adversaries increasingly rely on legitimate system tools, living-off-the-land techniques, and credential abuse to evade traditional security controls. As a result, organizations must shift from reactive alert monitoring toward proactive, behavior-based detection strategies.

Detection Engineering is the discipline of designing, developing, testing, and maintaining security detections using structured engineering practices. Rather than relying on static alerts or ad hoc detection creation, detection engineering treats detections as managed artifacts that follow a defined lifecycle, including research, development, validation, deployment, and continuous improvement.

This proposal outlines the implementation of a **Detection Engineering Program** designed to improve the organization's ability to identify and respond to adversary activity across enterprise systems and networks. The program introduces a structured detection development lifecycle aligned with industry frameworks such as **MITRE ATT&CK**, enabling security teams to systematically detect adversary tactics, techniques, and procedures (TTPs).

The proposed program will introduce several key capabilities:

- A standardized **Detection Engineering Lifecycle** for developing and maintaining detection rules
  - **Threat-informed detection development** aligned with adversary behavior models
  - **Detection-as-Code practices** using version-controlled rule definitions
  - **Continuous validation through adversary simulation and purple-team exercises**
  - **Operational feedback loops** to tune detections and reduce false positives
  - **Metrics and reporting** to measure detection coverage and program effectiveness

By implementing this program, the Security Operations Center (SOC) will gain a more structured and scalable approach to detection development. The program will enable security teams to better identify emerging threats, improve detection accuracy, and reduce alert fatigue through continuous tuning and validation.

Ultimately, the Detection Engineering Program will strengthen the organization's cybersecurity posture by ensuring that detection capabilities evolve alongside the threat landscape and remain aligned with operational security objectives.

Introduction

Detection Engineering is the discipline of designing, developing, testing, deploying, and maintaining security detections through structured and repeatable engineering practices. Rather than relying on static alerts or ad hoc detection creation, detection engineering treats detections as managed artifacts that follow a defined lifecycle and are continuously improved over time.

As modern adversaries increasingly rely on stealth, legitimate administrative tools, credential abuse, and living-off-the-land techniques, organizations must adopt more deliberate and threat-informed methods for identifying malicious activity. Traditional monitoring approaches alone are no longer sufficient to provide the visibility and detection fidelity required to support a mature Security Operations Center (SOC).

This proposal outlines the establishment of a **Detection Engineering Program** to improve the organization's ability to identify, validate, and operationalize high-quality detections across enterprise systems and networks. The program is intended to provide a formal structure for how detections are requested, researched, developed, tested, deployed, tuned, and maintained.

The proposed program integrates key stakeholders across the security organization, including the SOC, threat intelligence, incident response, and red or purple team functions. By bringing these inputs into a common lifecycle, the organization can ensure that detection content is aligned with real-world adversary behavior, operational priorities, and the evolving threat landscape.

This document presents the recommended framework, lifecycle, governance model, metrics, and implementation approach for standing up a scalable Detection Engineering capability. The objective is to move from reactive alert management toward a proactive, threat-informed, and continuously improving detection program that strengthens overall cybersecurity operations.

Why a Detection Engineering Program Is Needed

Organizations today face an increasingly complex and rapidly evolving cybersecurity threat landscape. Adversaries are continuously adapting their tactics, techniques, and procedures to evade traditional security controls. Modern attacks frequently leverage legitimate system tools, compromised credentials, and stealth techniques that blend into normal system activity, making detection significantly more difficult.

At the same time, Security Operations Centers (SOCs) are responsible for monitoring a growing volume of security telemetry generated from endpoints, identity platforms, cloud services, and network infrastructure. Without a structured approach to detection development, this expanding data environment can result in an overwhelming number of alerts, inconsistent detection coverage, and difficulty identifying meaningful threats among routine activity.

Many organizations rely heavily on vendor-provided alerts or reactive rule creation following security incidents. While these detections provide baseline visibility, they often do not fully address organization-specific risks, emerging adversary behaviors, or gaps in monitoring coverage. As a result, security teams may experience high false positive rates, alert fatigue, and limited visibility into sophisticated attacker activity.

A Detection Engineering Program addresses these challenges by introducing a structured and proactive approach to detection development. Rather than treating alerts as static configurations, detection engineering establishes a lifecycle for how detections are researched, developed, tested, deployed, and continuously improved. This process enables security teams to systematically align detection logic with adversary behaviors and operational priorities.

By adopting detection engineering practices, the organization can improve the quality and reliability of its detection capabilities, reduce operational noise, and ensure that monitoring strategies evolve alongside the threat landscape. The program also creates a framework for collaboration between SOC analysts, threat intelligence teams, incident responders, and red or purple team operators, allowing insights from real-world investigations and simulations to directly inform detection development.

Implementing a formal Detection Engineering Program ultimately strengthens the organization’s defensive posture by improving visibility into adversary activity, reducing detection gaps, and enabling a more proactive and resilient security operations capability.

Goals of the Program

The Detection Engineering Program is intended to establish a structured and scalable approach to developing, validating, and maintaining high-quality security detections across the organization's technology environment. The program focuses on improving the effectiveness of security monitoring while reducing operational inefficiencies within the Security Operations Center (SOC).

The primary goals of the program include:

• Establish a **standardized Detection Engineering Lifecycle** for the research, development, testing, deployment, and maintenance of detection rules.

• Implement **threat-informed detection development** aligned with adversary behaviors and frameworks such as MITRE ATT&CK.

• Improve the **accuracy and reliability of detections** to reduce false positives and minimize alert fatigue within the SOC.

• Introduce **Detection-as-Code practices** to manage detection logic using version-controlled rule definitions and structured development workflows.

• Enable **continuous validation of detections** through adversary simulation, purple-team exercises, and operational feedback.

• Improve **visibility into detection coverage** across critical adversary tactics and techniques.

• Establish **metrics and reporting capabilities** to measure detection effectiveness and program maturity.

• Foster **collaboration between SOC analysts, threat intelligence, incident response, and red or purple team functions** to ensure detection development reflects real-world threats.

• Provide a **repeatable and scalable framework** that allows the detection program to evolve alongside the organization's infrastructure and the changing threat landscape.

By achieving these objectives, the Detection Engineering Program will strengthen the organization's ability to detect malicious activity earlier in the attack lifecycle while supporting a more efficient and resilient security operations capability.

Detection Engineering Frameworks and Standards

The Detection Engineering Program will leverage established industry frameworks, standards, and methodologies to guide the development, validation, and continuous improvement of detection capabilities. These frameworks provide structured models for understanding adversary behavior, evaluating defensive capabilities, and ensuring detections align with recognized cybersecurity best practices.

By adopting these frameworks, the organization can ensure that detection development is both threat-informed and aligned with widely accepted security methodologies. These frameworks also enable more consistent communication across security teams and provide a common reference point for evaluating detection coverage and program maturity.

Adversary Behavior Frameworks

The program will utilize adversary behavior frameworks to ensure detections are aligned with real-world attacker tactics and techniques.

MITRE ATT&CK
The MITRE ATT&CK framework provides a comprehensive knowledge base of adversary tactics, techniques, and procedures (TTPs) observed in real-world cyber operations. Detection development will be mapped to ATT&CK techniques to ensure visibility across key stages of the attack lifecycle and to identify potential gaps in detection coverage.

MITRE D3FEND
MITRE D3FEND complements the ATT&CK framework by providing a catalog of defensive techniques and countermeasures. This framework helps security teams identify defensive strategies that can be used to detect, mitigate, or disrupt adversary activity.

MITRE Engage
MITRE Engage provides guidance on adversary engagement strategies and defensive techniques designed to increase adversary cost and reduce attacker effectiveness. While primarily focused on defensive engagement, the framework also provides useful insight into adversary behavior patterns that can inform detection development.

Detection Engineering Methodologies

Several detection engineering methodologies will guide the structure and maturity of the program.

Detection Engineering Behavior Maturity Model (DEBMM)
The Detection Engineering Behavior Maturity Model provides a framework for evaluating the maturity of an organization's detection capabilities. This model helps organizations assess their current detection engineering practices and define a roadmap for improving detection coverage, validation processes, and operational integration.

Detection Maturity Matrix (DMM)
The Detection Maturity Matrix provides a structured model for evaluating the quality and effectiveness of detection logic across multiple dimensions, including detection fidelity, coverage, and operational readiness.

Threat Detection Maturity Framework
This framework provides guidance for assessing detection capabilities and measuring improvements as detection engineering practices evolve.

Detection Development Frameworks

Detection development will follow structured methodologies that support repeatable detection creation and validation.

Detection-as-Code (DaC)
Detection-as-Code applies software engineering principles to detection development by treating detection logic as version-controlled artifacts. This approach enables structured rule development, code review processes, automated testing, and CI/CD deployment pipelines.

DeTT&CT Framework
The DeTT&CT (Detection, Threat Hunting, and Coverage Testing) framework provides tooling and methodology for mapping detection logic to MITRE ATT&CK techniques and evaluating telemetry coverage across security monitoring platforms.

Security Standards and Guidance

In addition to detection-specific frameworks, the program will align with widely recognized cybersecurity standards and operational guidance.

NIST SP 800-61 – Computer Security Incident Handling Guide
Provides guidance for incident response processes and coordination across security teams.

NIST SP 800-92 – Guide to Computer Security Log Management
Defines best practices for collecting, managing, and analyzing security log data.

NIST SP 800-86 – Guide to Integrating Forensic Techniques into Incident Response
Provides guidance on using digital forensic techniques during security investigations.

By aligning the Detection Engineering Program with these frameworks and standards, the organization can ensure that detection development follows proven methodologies and supports broader cybersecurity governance and operational objectives.

Detection Engineering Lifecycle

The Detection Engineering Lifecycle defines the structured process used to design, develop, validate, deploy, and maintain security detections within the Security Operations Center (SOC). Rather than creating detection rules in an ad hoc or reactive manner, the lifecycle provides a repeatable framework that ensures detections are aligned with adversary behaviors, operational priorities, and available telemetry.

This lifecycle enables security teams to continuously improve detection coverage by incorporating intelligence insights, incident response findings, and adversary simulation results. Each phase of the lifecycle supports collaboration across security teams and ensures detection logic remains effective as both the organization's infrastructure and the threat landscape evolve.

The Detection Engineering Lifecycle consists of the following phases:

1. Detection Request and Intake

Detection development typically begins with the identification of a new detection opportunity. Requests may originate from several sources, including threat intelligence reports, incident response investigations, SOC analyst observations, red or purple team exercises, vulnerability disclosures, or newly observed adversary techniques.

During the intake phase, detection requests are documented and prioritized based on risk, relevance to the organization's environment, and potential impact on security monitoring capabilities.

2. Research and Analysis

Once a detection opportunity has been identified, detection engineers conduct research to understand the adversary behavior associated with the request. This phase includes reviewing threat intelligence, analyzing relevant MITRE ATT&CK techniques, identifying applicable telemetry sources, and evaluating how the activity may manifest within the organization's monitoring platforms.

The goal of this phase is to determine whether sufficient telemetry exists to detect the behavior and to identify the most reliable indicators that can be used to build detection logic.

3. Detection Development

During the development phase, detection engineers translate research findings into actionable detection logic. This typically involves creating queries, rules, or analytics designed to identify suspicious behavior within security monitoring platforms such as a Security Information and Event Management (SIEM) system.

Detection logic is developed using standardized templates and detection-as-code practices, enabling rule definitions to be version-controlled, reviewed, and maintained within a structured repository.

4. Validation and Testing

Before deployment, detections must be validated to ensure they accurately identify malicious behavior without generating excessive false positives. Testing may involve adversary simulation tools, purple team exercises, or replaying historical telemetry to evaluate detection performance.

This validation process ensures detections are operationally effective before being introduced into the production monitoring environment.

5. Deployment

Once validated, detection rules are deployed into the production security monitoring environment. Deployment may occur manually or through automated CI/CD pipelines depending on the maturity of the detection engineering program.

During this phase, documentation is finalized and the SOC is informed of new detection capabilities.

6. Monitoring and Tuning

After deployment, detections are continuously monitored to evaluate their performance in the operational environment. SOC analysts provide feedback regarding alert quality, false positives, and potential improvements.

Detection engineers use this feedback to refine detection logic, adjust thresholds, and improve rule reliability.

7. Continuous Improvement and Maintenance

Detection engineering is an ongoing process. As adversaries evolve their techniques and organizational infrastructure changes, detections must be regularly reviewed and updated. Continuous improvement ensures that the detection program remains aligned with emerging threats and maintains effective visibility across the environment.

Detection Engineering Operating Model

The Detection Engineering Operating Model defines how detection development activities are coordinated across the security organization. While detection engineers are responsible for building and maintaining detection logic, effective detection engineering requires collaboration between multiple security functions, including the Security Operations Center (SOC), threat intelligence, incident response, and red or purple team operations.

The operating model is designed to ensure that detection development is informed by real-world threats, operational experience, and adversary simulation. By integrating these inputs into the detection lifecycle, the organization can continuously improve its ability to detect malicious activity and respond to emerging threats.

Within this model, detection engineers serve as the primary owners of detection development and maintenance. However, detection opportunities and operational insights may originate from several sources across the security organization.

Security Operations Center (SOC)

SOC analysts play a critical role in identifying detection gaps and providing operational feedback on existing detections. Through alert triage, investigations, and threat monitoring activities, SOC personnel frequently observe patterns of activity that may indicate opportunities for improved detection logic.

SOC feedback is particularly valuable for identifying false positives, refining detection thresholds, and ensuring alerts remain actionable within the operational environment.

Threat Intelligence

Threat intelligence teams provide insight into emerging adversary tactics, techniques, and procedures (TTPs). These insights help guide detection development by identifying new behaviors that should be monitored within the organization's environment.

Threat intelligence reporting often serves as an early indicator of evolving attacker methodologies, enabling detection engineers to proactively develop detections before those techniques are observed within the organization.

Incident Response

Incident response teams provide valuable insights into adversary behavior observed during active investigations. Post-incident analysis frequently reveals new detection opportunities based on attacker techniques that were not previously monitored or fully understood.

Incorporating lessons learned from incident response activities helps ensure the detection program evolves in response to real-world security events.

Red Team and Purple Team Operations

Red team and purple team exercises provide controlled adversary simulations that help validate detection capabilities. These exercises allow the organization to test whether existing detections successfully identify simulated attacker behavior.

Results from these exercises are used to identify detection gaps and improve the reliability of existing detection logic.

Detection Engineering

Detection engineers are responsible for translating operational insights into actionable detection logic. This includes researching adversary behaviors, developing detection rules, validating their effectiveness, and deploying them into the monitoring environment.

Detection engineers also maintain detection documentation, manage rule repositories, and ensure detection logic remains aligned with organizational monitoring priorities.

By integrating these functions into a collaborative operating model, the Detection Engineering Program ensures that detection development remains closely aligned with both the threat landscape and the operational needs of the SOC.

Detection Engineering Governance (RACI)

Effective detection engineering requires clear governance to ensure that detection development, deployment, and maintenance activities are performed consistently and in alignment with organizational security objectives. Governance provides structure around how detections are requested, developed, approved, deployed, and maintained over time.

To support this structure, the Detection Engineering Program adopts a **RACI governance model**, which defines roles and responsibilities across participating security teams. The RACI framework identifies who is **Responsible**, **Accountable**, **Consulted**, and **Informed** for key activities within the detection lifecycle.

This governance approach helps ensure that detection engineering activities are properly coordinated across the organization and that ownership of detection processes is clearly defined.

Responsible

Individuals or teams responsible for executing specific detection engineering tasks. This typically includes detection engineers who design, develop, validate, and maintain detection rules.

Accountable

The individual or leadership role ultimately accountable for ensuring detection engineering activities are completed and aligned with organizational priorities. This role typically resides within the security leadership or SOC management structure.

Consulted

Teams that provide input, expertise, or operational feedback during the detection development process. These groups may include SOC analysts, threat intelligence analysts, incident response teams, and red or purple team personnel.

Informed

Stakeholders who must be notified of detection changes, updates, or program developments. These individuals may include security leadership, platform owners, and other operational teams that rely on detection outputs.

Establishing clearly defined governance ensures that detection development remains coordinated, transparent, and aligned with operational security priorities. The detailed RACI matrix outlining responsibilities for detection engineering activities is provided in **Appendix E: Detection Engineering RACI Matrix**.

Detection Engineering Metrics and KPIs

To ensure the effectiveness and maturity of the Detection Engineering Program, a defined set of metrics and key performance indicators (KPIs) will be used to measure the performance, coverage, and operational impact of detection capabilities. These metrics provide visibility into the effectiveness of detection engineering efforts and allow leadership to evaluate how well the organization is identifying and responding to malicious activity.

Metrics also support continuous improvement by highlighting detection gaps, identifying opportunities for optimization, and measuring the operational impact of detection rules within the Security Operations Center (SOC).

The Detection Engineering Program will track several categories of metrics, including detection coverage, rule performance, operational impact, and program maturity.

Detection Coverage Metrics

Detection coverage metrics measure the organization's ability to detect adversary behaviors across known attack techniques and tactics. These metrics help identify gaps in detection visibility and guide future detection development efforts.

Examples include:

• Percentage of **MITRE ATT&CK techniques covered by existing detections**
• Coverage of high-risk adversary techniques relevant to the organization
• Number of new detections developed per quarter

Detection Quality Metrics

Detection quality metrics evaluate how effectively detection rules identify malicious behavior while minimizing unnecessary alerts.

Examples include:

• False positive rate for detection rules
• True positive detection rate during incident investigations
• Alert fidelity and analyst confidence levels

Operational Metrics

Operational metrics measure how detection rules impact SOC operations and how efficiently detections move through the development lifecycle.

Examples include:

• Average time to develop and deploy new detection rules
• Number of detections tuned or improved based on SOC feedback
• Mean time to detection (MTTD) for identified threats

Validation Metrics

Validation metrics measure the effectiveness of detections when tested against simulated adversary activity.

Examples include:

• Detection success rate during purple team exercises
• Number of detection gaps identified during adversary simulations
• Percentage of detections validated through testing activities

Program Maturity Metrics

Program maturity metrics provide insight into the overall development and evolution of the detection engineering capability.

Examples include:

• Number of detections managed through version-controlled repositories
• Percentage of detections validated through testing processes
• Growth of detection coverage over time

By tracking these metrics, the Detection Engineering Program can provide leadership with measurable insights into detection effectiveness, operational performance, and the overall maturity of the organization's security monitoring capabilities.

Implementation Timeline and Roadmap

The implementation of the Detection Engineering Program will follow a phased approach designed to gradually introduce processes, tooling, and governance structures without disrupting existing security operations. This phased model allows the organization to establish foundational capabilities first while progressively expanding detection coverage and operational maturity.

The roadmap focuses on building sustainable processes that enable long-term detection engineering success while aligning with the operational needs of the Security Operations Center (SOC).

Phase 1: Program Foundation

The first phase focuses on establishing the foundational structure required to support detection engineering activities. This includes defining governance, documentation standards, and the detection lifecycle framework.

Key activities during this phase include:

• Establishing the Detection Engineering Lifecycle and governance structure
• Defining detection rule templates and development standards
• Creating a centralized repository for detection rules
• Identifying priority adversary techniques and initial detection targets
• Establishing collaboration processes between SOC, threat intelligence, and incident response teams

The outcome of this phase is a formalized detection engineering framework and governance structure that can support consistent detection development.

Phase 2: Detection Development and Coverage Expansion

The second phase focuses on developing new detections and improving visibility into adversary activity across the environment. Detection engineering efforts during this phase will prioritize high-risk adversary behaviors and techniques relevant to the organization's threat landscape.

Key activities during this phase include:

• Developing new detections aligned with MITRE ATT&CK techniques
• Expanding telemetry coverage across priority data sources
• Implementing detection-as-code practices for rule management
• Establishing detection validation processes
• Improving alert quality through rule tuning and SOC feedback

The outcome of this phase is an expanded set of operational detections that improve the organization's ability to identify malicious activity.

Phase 3: Validation and Adversary Simulation

The third phase focuses on validating the effectiveness of detection capabilities through adversary simulation and purple team exercises. These exercises help confirm that detection rules successfully identify real-world attack behaviors.

Key activities during this phase include:

• Conducting adversary simulation exercises
• Performing purple team testing against detection rules
• Identifying detection gaps and areas for improvement
• Refining detection logic based on testing outcomes

The outcome of this phase is increased confidence in the organization's detection capabilities and improved resilience against adversary techniques.

Phase 4: Continuous Improvement and Program Maturity

The final phase focuses on sustaining and improving the detection engineering capability over time. Detection development becomes an ongoing process driven by threat intelligence, operational feedback, and evolving adversary behaviors.

Key activities during this phase include:

• Expanding detection coverage across additional attack techniques
• Continuously tuning detection rules based on operational feedback
• Monitoring detection metrics and program performance
• Maintaining detection documentation and rule repositories
• Integrating detection engineering into broader security operations processes

Through this phased implementation approach, the organization can establish a mature detection engineering capability that evolves alongside the threat landscape and the organization's security monitoring requirements.

Detection-as-Code Framework Recommendation

To support scalable and maintainable detection development, this proposal recommends adopting a **Detection-as-Code (DaC)** approach for managing detection rules and analytics.

Detection-as-Code applies modern software development practices to detection engineering by treating detection rules as structured code artifacts rather than static configurations within security tools. This approach allows detection logic to be version controlled, peer reviewed, tested, and maintained in a consistent and transparent manner.

By managing detection rules as code, the organization can establish standardized workflows for developing, validating, and deploying detections while maintaining a clear record of changes over time.

Detection Rule Standardization

Detection rules should be developed using standardized templates that define consistent metadata and rule structure. Standardized templates ensure that detection rules include required information such as detection logic, associated adversary techniques, rule severity, and known false positive conditions.

Standardization improves rule quality and ensures that detection logic remains understandable and maintainable across the security team.

Version Control

Detection rules should be stored in a centralized version-controlled repository. Version control enables detection engineers to track rule changes, maintain historical records of rule modifications, and collaborate more effectively on detection development.

Version control also allows teams to revert changes if necessary and provides visibility into how detection rules evolve over time.

Peer Review and Quality Assurance

Detection-as-Code workflows allow detection rules to undergo peer review prior to deployment. Peer review improves rule quality by ensuring that detection logic is accurate, efficient, and aligned with detection engineering standards.

This process helps reduce configuration errors and ensures that detection rules are validated before being introduced into production monitoring environments.

Detection Rule Lifecycle Management

Detection-as-Code also enables the implementation of a structured rule lifecycle that reflects the maturity of each detection rule. Detection rules typically progress through several stages:

• **Experimental** – initial rule development and early testing
• **Testing** – validated rule undergoing operational evaluation
• **Production** – stable rule actively used for security monitoring

Lifecycle tracking ensures that detection rules are appropriately validated before they are relied upon in operational environments.

Integration with Security Monitoring Platforms

While detection rules are managed within a version-controlled repository, they ultimately support security monitoring platforms such as Security Information and Event Management (SIEM) systems and endpoint detection platforms. Detection-as-Code workflows provide a structured process for promoting detection rules from development environments into production monitoring systems.

This integration ensures that detection logic can be consistently deployed while maintaining governance and change tracking.

By adopting Detection-as-Code practices, the organization can significantly improve the consistency, transparency, and scalability of its detection engineering program while aligning detection development with modern engineering practices.

Detection Engineering Maturity Model Assessment

The Detection Engineering Program will evolve over time as processes, tooling, and operational collaboration mature. To support this growth, the organization can evaluate the program using a maturity model that measures the progression of detection capabilities from reactive monitoring toward proactive, threat-informed detection engineering.

A maturity model provides leadership with a structured way to evaluate current capabilities, identify areas for improvement, and track the long-term development of the detection program.

Detection engineering maturity can generally be described across several levels of capability.

Level 1: Reactive Detection

At the initial stage of maturity, detection activities are largely reactive and driven by individual incidents or vendor-provided alerts. Detection logic is often developed in response to specific security events rather than as part of a structured detection strategy.

At this stage, detection rules may be inconsistent, poorly documented, and difficult to maintain. Limited governance and testing processes may exist, resulting in higher false positive rates and reduced visibility into adversary activity.

Level 2: Structured Detection Development

As detection engineering capabilities begin to mature, organizations establish more structured processes for detection development. Detection rules are created using defined templates, and development begins to align with recognized adversary behaviors such as those documented in the MITRE ATT&CK framework.

At this stage, detection engineers begin to collaborate more closely with SOC analysts, threat intelligence teams, and incident response personnel to improve detection quality and coverage.

Level 3: Threat-Informed Detection Engineering

At this level of maturity, detection development is consistently aligned with adversary tactics, techniques, and procedures. Detection engineering processes become more proactive, focusing on identifying detection gaps before they are exploited by attackers.

Threat intelligence and adversary simulation exercises play a significant role in guiding detection development. Detection rules are routinely tested and validated against simulated attack scenarios.

Level 4: Continuous Detection Validation

Highly mature detection engineering programs continuously validate detection capabilities through regular testing and adversary simulation. Purple team exercises, red team assessments, and attack emulation tools are used to ensure detection logic successfully identifies malicious activity.

Detection performance metrics are regularly reviewed, and detection rules are continuously refined based on operational feedback and testing outcomes.

Level 5: Mature Detection Engineering Program

At the highest level of maturity, detection engineering becomes a fully integrated component of the organization's security operations strategy. Detection rules are managed using detection-as-code practices, enabling structured development, version control, and automated deployment.

Detection coverage is regularly assessed against adversary frameworks, and the organization maintains strong visibility into its ability to detect malicious activity across the enterprise environment.

By adopting a maturity model approach, the organization can establish clear goals for improving detection capabilities while ensuring that the Detection Engineering Program evolves in a structured and measurable way.

Priority Data Sources for Detection Engineering

Effective detection engineering depends on the availability of reliable and comprehensive telemetry from across the organization's technology environment. Security monitoring platforms rely on these data sources to identify suspicious behavior, correlate activity across systems, and detect adversary tactics and techniques.

To support the Detection Engineering Program, priority should be placed on integrating and maintaining visibility across key security telemetry sources. These data sources provide the behavioral signals necessary for detecting malicious activity across endpoints, identities, networks, and cloud services.

The following categories represent high-value telemetry sources that are critical for effective detection engineering.

Endpoint Telemetry

Endpoint telemetry provides visibility into activity occurring on user workstations and servers. This includes process execution, file activity, registry modifications, and system events that may indicate malicious behavior.

Endpoint monitoring platforms often generate high-fidelity telemetry that can be used to detect techniques such as privilege escalation, credential access, and persistence mechanisms.

Examples include:

• Process creation and command-line activity
• File system modifications
• Registry changes
• Endpoint security alerts

Identity and Authentication Logs

Identity-related telemetry provides insight into authentication activity and user behavior across enterprise systems. Monitoring identity activity is critical for detecting credential misuse, account compromise, and lateral movement attempts.

Examples include:

• Authentication attempts and sign-in activity
• Account creation and modification events
• Privileged account usage
• Directory service activity

Network Telemetry

Network telemetry provides visibility into communication between systems and external entities. Network activity can reveal indicators of command-and-control activity, data exfiltration attempts, and suspicious outbound connections.

Examples include:

• Network connection events
• DNS queries and responses
• Firewall and proxy logs
• Network intrusion detection alerts

Cloud and SaaS Activity

As organizations increasingly rely on cloud services, visibility into cloud platforms and software-as-a-service (SaaS) applications becomes essential for detection engineering.

Monitoring cloud activity allows security teams to detect suspicious administrative actions, unauthorized access attempts, and abnormal data access patterns.

Examples include:

• Cloud platform activity logs
• Administrative actions in cloud services
• Data access and sharing events
• Application access activity

Security Platform Alerts

Security tools such as endpoint protection platforms, identity protection systems, and threat detection platforms generate alerts based on built-in analytics. These alerts can provide valuable signals that support detection engineering and correlation within security monitoring platforms.

Examples include:

• Endpoint detection alerts
• Identity protection alerts
• threat intelligence alerts
• vulnerability or exploitation alerts

By prioritizing visibility across these telemetry sources, the Detection Engineering Program can ensure that detection rules are supported by the data required to identify adversary activity across the enterprise environment.

Data Onboarding Strategy

A successful Detection Engineering Program depends on the availability of high-quality telemetry from across the organization's technology environment. As new systems, applications, and platforms are introduced, security telemetry must be evaluated and onboarded in a structured manner to ensure that relevant security events are available for monitoring and detection.

The Data Onboarding Strategy defines a consistent process for evaluating, integrating, and operationalizing new data sources within the organization's security monitoring environment. This process ensures that telemetry sources are prioritized based on their value for detecting adversary behavior and supporting security investigations.

Identification of New Data Sources

New telemetry sources may be identified through several channels, including system deployments, new cloud services, security tool integrations, and recommendations from security teams. During this phase, security engineers evaluate whether a data source provides relevant security visibility and whether it can support detection use cases.

Examples of candidate data sources include new endpoint platforms, identity systems, network monitoring tools, cloud services, and application logs.

Data Source Evaluation

Once a potential data source is identified, the telemetry is evaluated to determine the types of security-relevant events it provides. This evaluation includes reviewing event fields, understanding logging configurations, and assessing the data's usefulness for identifying adversary activity.

The evaluation process helps determine whether the data source provides sufficient visibility to support detection engineering use cases.

Integration into Security Monitoring Platforms

After evaluation, the data source is integrated into the organization's security monitoring platform, such as a Security Information and Event Management (SIEM) system. During this phase, ingestion pipelines, parsing logic, and data normalization processes are configured to ensure the telemetry can be effectively queried and analyzed.

Proper normalization and field mapping are critical to ensuring that detection rules can reliably reference relevant event data.

Detection Use Case Development

Once telemetry is available within the monitoring platform, detection engineers identify potential detection opportunities associated with the new data source. These use cases are aligned with adversary techniques and organizational security priorities.

Detection rules are then developed and tested using the newly onboarded telemetry to ensure that the data source contributes meaningful visibility to the detection program.

Continuous Monitoring and Optimization

After onboarding, telemetry sources are continuously monitored to ensure that data ingestion remains stable and that detection rules continue to function as expected. Changes to system configurations, logging formats, or platform updates may require adjustments to parsing logic or detection queries.

Regular review of data sources ensures that the detection program maintains reliable visibility across the organization's technology environment.

By following a structured onboarding strategy, the Detection Engineering Program can ensure that new telemetry sources are integrated in a way that directly supports detection capabilities and enhances the organization's overall security monitoring posture.

Purple Team Validation Plan

To ensure that detection rules effectively identify adversary behavior, the Detection Engineering Program incorporates a structured validation process using purple team exercises and adversary simulation activities.

Purple teaming combines offensive and defensive security practices by bringing together red team operators, detection engineers, and SOC analysts to collaboratively test detection capabilities. These exercises simulate real-world attack techniques in a controlled environment and allow the organization to evaluate whether existing detections successfully identify malicious activity.

Through this process, the organization can validate the effectiveness of detection logic, identify detection gaps, and continuously improve detection coverage.

Detection Validation Through Adversary Simulation

Adversary simulation tools and testing frameworks can be used to emulate known attacker techniques mapped to the MITRE ATT&CK framework. These simulations generate telemetry that mimics real adversary behavior, allowing detection engineers to verify whether detection rules trigger as expected.

By testing detections against simulated attacks, the organization can confirm that monitoring systems are capable of identifying relevant adversary tactics, techniques, and procedures.

Collaborative Purple Team Exercises

Purple team exercises involve coordinated collaboration between offensive security personnel and defensive teams. During these exercises, red team operators simulate attacker behaviors while detection engineers and SOC analysts observe the resulting telemetry and evaluate detection effectiveness.

These exercises provide valuable insight into how attackers interact with the environment and whether detection rules provide adequate visibility into those activities.

Identification of Detection Gaps

Purple team activities frequently reveal areas where detection coverage is incomplete or insufficient. When detection gaps are identified, detection engineers develop new detection rules or enhance existing ones to address those gaps.

This process ensures that detection capabilities continuously evolve to address emerging attacker techniques.

Continuous Improvement of Detection Rules

Results from adversary simulations and purple team exercises are documented and incorporated into the detection engineering lifecycle. Detection engineers use these findings to refine detection logic, improve alert fidelity, and expand detection coverage across the enterprise environment.

By integrating validation exercises into the detection engineering process, the organization ensures that detection capabilities remain aligned with real-world attacker behaviors and provide meaningful protection against evolving threats.

Appendix A – Detection Rule Request Template

This appendix provides the standardized template used to request new detection rules within the Detection Engineering Program. The template ensures that detection requests contain sufficient context, threat intelligence references, and technical details to support efficient rule development and prioritization.

Detection requests may originate from SOC analysts, threat intelligence teams, incident response personnel, or security engineering staff. Using a consistent request format ensures that detection engineers can properly evaluate the request and determine the feasibility of developing a corresponding detection rule.

The following template should be used when submitting new detection rule requests.

Rule Request Template

```
## Rule Request Template
---
name: New Detection Rule Request
about: Suggest a new detection rule for Microsoft Sentinel
title: "[Rule Request] <short description>"
labels: enhancement, detection-rule
assignees: aring87

---

## Summary

Describe the detection idea:

What behavior or attack is this rule meant to detect?

What log source(s) does it depend on?

---

## MITRE ATT&CK Mapping

| Tactic | Technique | Technique ID |
|------|------|------|
| e.g., Execution | PowerShell | T1059.001 |
| e.g., Credential Access | LSASS Memory Dump | T1003.001 |

---
```

Context & Use Case

Why is this detection valuable?

Any relevant threat actor, malware, or CVE associations?

Sample Logs or KQL

(Optional) Paste sample KQL or describe expected fields here.

Lifecycle

- [ ] experimental
- [ ] testing
- [ ] production

Related Issues or References

Link to threat intel, related alerts, blog posts, etc.

Appendix B – YAML Detection Rule Template

This appendix provides the standardized YAML template used for developing detection rules within the Detection Engineering Program. The template supports Detection-as-Code practices by defining a consistent rule structure that includes detection logic, metadata, lifecycle state, and mappings to adversary techniques.

Using a standardized rule format improves rule consistency, supports version control, and enables detection engineers to manage detection rules as structured artifacts within a centralized repository.

The template also supports mapping detections to frameworks such as **MITRE ATT&CK**, allowing the organization to track detection coverage across adversary tactics and techniques.

The following YAML template represents the standard structure for detection rules used within the Detection Engineering Program.

YAML Detection Rule Template

```
title: [Short descriptive rule name]
id: [unique-rule-id]  # e.g., sigma-031
status: experimental  # or: testing, production

description: |
  [Brief explanation of what the rule detects and why it matters.]

author: [your.name or team name]
date: [yyyy-mm-dd]

tags:
  - attack.txxxx
  - attack.[tactic]
  - killchain.[stage]
  - [custom tags]

logsource:
  product: windows
  category: process_creation

detection:
  selection:
    query: |
      [Insert KQL here]

      Example:
      DeviceProcessEvents
```

```
      | where FileName has "mimikatz"
      | project TimeGenerated, DeviceName, AccountName, FileName,
ProcessCommandLine

  condition: selection

falsepositives:
  - [Optional explanation of known benign triggers]

level: high

tactics:
  - Credential Access
  - Discovery

techniques:
  - T1003.001
  - T1087.002

lifecycle: experimental
```

Appendix C – Validation Log Template

This appendix provides the standardized template used to document detection validation activities within the Detection Engineering Program. Validation logs capture the results of detection testing conducted through adversary simulations, purple team exercises, or controlled testing scenarios.

Maintaining a structured validation log ensures that detection rules are tested against known adversary techniques and that the results of those tests are documented for future reference. These records support continuous improvement by identifying detection gaps, validating rule effectiveness, and documenting changes made to detection logic over time.

The following template can be used to record detection validation activities.

Detection Validation Log Template

Field	Description
Validation Date	Date the detection rule was tested
Detection Rule Name	Name of the detection rule being validated
Detection Rule ID	Unique identifier for the detection rule
Detection Lifecycle Stage	Experimental, Testing, or Production
Adversary Technique Tested	MITRE ATT&CK technique used during testing
Simulation Method	Method used for validation (purple team exercise, adversary simulation tool, manual test, etc.)
Data Source	Telemetry source used to validate the detection
Detection Result	Successful detection, partial detection, or detection failure
Alert Triggered	Yes / No
False Positives Observed	Description of any false positives
Detection Gaps Identified	Description of any observed gaps
Remediation Actions	Detection improvements or tuning performed
Analyst Notes	Additional observations from validation testing

---

Example Entry

Field	Example
Validation Date	2026-03-01
Detection Rule Name	Suspicious LSASS Access
Detection Rule ID	sigma-041
Detection Lifecycle Stage	Testing
Adversary Technique Tested	T1003.001 – LSASS Credential Dumping
Simulation Method	Atomic Red Team
Data Source	Endpoint Process Telemetry
Detection Result	Successful Detection
Alert Triggered	Yes
False Positives Observed	None observed during testing
Detection Gaps Identified	N/A
Remediation Actions	Detection promoted to production
Analyst Notes	Detection triggered within expected timeframe

Appendix D – Change Control and Version History

This appendix documents how updates to the Detection Engineering Program and its associated artifacts are tracked over time. Maintaining a version history ensures that changes to the program framework, detection templates, and supporting processes are recorded in a consistent and transparent manner.

Change control provides accountability by documenting when modifications occur, what changes were made, and who approved those changes. This process helps ensure that updates to the detection engineering framework remain aligned with organizational security objectives and operational requirements.

Version history tracking is particularly important for maintaining documentation integrity and supporting audit or compliance reviews.

Document Version History

Version	Date	Author	Description of Changes
1.0	2025-XX-XX	Adam Ring	Initial draft of Detection Engineering Program proposal
2.0	2026-XX-XX	Adam Ring	Updated lifecycle model and governance framework
3.0	2026-XX-XX	Adam Ring	Expanded program documentation and appendices

---

Change Control Log Template

The following template can be used to document significant updates to detection engineering processes, documentation, or detection rule frameworks.

Change Date	Change Description	Affected Section	Author	Approval
YYYY-MM-DD	Description of update	Section name or appendix	Author name	Approver name

---

Example Entry

Change Date	Change Description	Affected Section	Author	Approval
2026-03-01	Added Purple Team Validation Plan	Section 15	Adam Ring	SOC Leadership

Detection Engineering RACI Matrix

Activity	Detection Engineering	SOC Analysts	Threat Intelligence	Incident Response	Security Leadership
Detection Request Intake	R	C	C	C	I
Detection Research	R	C	R	C	I
Detection Rule Development	R	C	C	C	I
Detection Validation Testing	R	C	C	R	I
Detection Deployment	R	C	I	I	A
Detection Monitoring and Tuning	R	R	C	C	I
Purple Team Validation	R	C	C	R	I
Detection Coverage Review	R	C	R	C	A
Detection Program Governance	R	I	I	I	A

RACI Definitions

Responsible (R)
The individual or team responsible for performing the work required to complete the activity.

Accountable (A)
The role ultimately accountable for ensuring the activity is completed and aligned with organizational objectives.

Consulted (C)
Individuals or teams whose input or expertise is required before the activity is completed.

Informed (I)
Stakeholders who are notified of progress or outcomes but are not directly involved in the activity.